

CSEN 296B-2 | Week 7.2 Lab Submission

AstraNotes Testing Strategy and First Test Set

Student Name: Alex Shirazi

Date: May 18, 2026

Project: AstraNotes

1. AstraNotes Testing Strategy

The testing strategy for AstraNotes is built on three principles drawn directly from the project requirements and design artifacts.

Principle 1: Test what the requirements say must be true

Every test must trace to at least one requirement ID from the Week 3.1 refined baseline. Tests that cannot be linked to a requirement are low-value noise and should not be written or maintained. This principle comes directly from SPR-03: all core components shall have at least one automated unit test verifying their primary behavior before each milestone delivery.

Principle 2: Test at the right level for the right reason

AstraNotes has three distinct testing levels, each with a specific purpose:

Unit tests verify one class or function in isolation using fakes or temporary directories. These are the fastest and most numerous tests. They run on every save or commit and catch logic errors immediately.

Integration tests verify that two or more components work correctly together: for example, that NoteService correctly routes a create call through ValidationLayer and into JsonFileRepository, and that the resulting file is readable. These run on every pull request or sprint boundary.

Feature tests verify an end-to-end user story from CLI input to persisted output. These are slower and more brittle. They run at milestone boundaries and before demo submissions, not on every commit.

Principle 3: Prioritize the highest-risk requirements first

The Week 5.2 traceability matrix identified NFR-01 (performance) as Weakly Traced and FR-04/SPR-01 (private note encryption) as the most architecturally significant requirement. The first test set focuses on FR-01 (note creation with validation) and FR-04/SPR-01 (SecureNote encryption) because these are the two features where a silent failure would be hardest to detect and most damaging.

Features Selected for First Test Set

Feature 1: Note Creation (FR-01, US-01) because it is the entry point for all other functionality. If create is broken, nothing else is testable. It also exercises the ValidationLayer, NoteService, and JsonFileRepository in sequence.

Feature 2: Private Note Encryption (FR-04, SPR-01, US-04) because it is the highest architectural risk in the system. A silent failure here means private notes are stored as plaintext on disk, which directly violates SPR-01. This must be caught by automated tests before any persistence slice is considered done.

2. First Test Set

Feature 1: Note Creation (FR-01, US-01)

These tests verify the complete create path: validation, object construction, persistence, and failure handling. All tests use pytest with tmp_path so no real file system state is required outside the test.

```
# tests/test_note_creation.py
# Requirement: FR-01, US-01
# Level: Unit (validate_title, Note dataclass) + Integration (NoteService + repo)

import pytest
from astranotes.models.note import Note, ValidationError, StoragePersistenceError
from astranotes.repository.json_file_repository import JsonFileRepository
from astranotes.service.note_service import NoteService
from astranotes.validation.validation_layer import ValidationLayer

@pytest.fixture
def service(tmp_path):
    return NoteService(JsonFileRepository(tmp_path), ValidationLayer())

# --- Unit: ValidationLayer ---

def test_empty_title_raises_validation_error(service):
    with pytest.raises(ValidationError):
        service.create(title="", body="Some body")

def test_whitespace_only_title_raises_validation_error(service):
    with pytest.raises(ValidationError):
        service.create(title=" ", body="Some body")

# --- Integration: NoteService + JsonFileRepository ---

def test_create_note_assigns_uuid(service):
    note = service.create(title="Test", body="Body")
    assert note.id is not None and len(note.id) == 36

def test_create_note_assigns_utc_timestamps(service):
    note = service.create(title="Test", body="")
    assert note.created_at.tzinfo is not None # UTC required (FR-06 refined)
    assert note.modified_at.tzinfo is not None

def test_create_note_persists_json_file(service, tmp_path):
    note = service.create(title="Saved", body="Check disk")
```

```

    assert (tmp_path / f"{note.id}.json").exists()

def test_create_note_body_can_be_empty(service):
    note = service.create(title="Title only", body="")
    assert note.body == "" # body is optional (FR-01 refined)

def test_create_note_tags_stored_as_entered(service):
    note = service.create(title="Tagged", body="", tags=["Python", "python"])
    assert note.tags == ["Python", "python"] # case-sensitive (FR-06)

```

Feature 2: Private Note Encryption (FR-04, SPR-01, US-04)

These tests verify that SecureNote never stores plaintext body content on disk, that decrypt reverses encrypt exactly, and that a failed decryption surfaces a clear error rather than returning garbled content.

```

# tests/test_secure_note.py
# Requirement: FR-04, SPR-01, US-04
# Level: Unit (SecureNote + PrivacyService) + Integration (SecureNote + repo)

import pytest
from cryptography.fernet import Fernet
from astranotes.models.note import Note, StoragePersistenceError
from astranotes.privacy.privacy_service import PrivacyService

@pytest.fixture
def key():
    return Fernet.generate_key() # test key only, never in source (SPR-01)

@pytest.fixture
def privacy(key):
    return PrivacyService(key)

# --- Unit: PrivacyService round-trip ---

def test_encrypt_decrypt_round_trip(privacy):
    original = "My secret note body"
    encrypted = privacy.encrypt(original)
    assert privacy.decrypt(encrypted) == original

def test_encrypted_output_is_not_plaintext(privacy):
    body = "sensitive content"
    encrypted = privacy.encrypt(body)
    assert body not in str(encrypted) # SPR-01: never stored as plaintext

```

```

def test_wrong_key_raises_storage_error():
    key_a = Fernet.generate_key()
    key_b = Fernet.generate_key()
    svc_a = PrivacyService(key_a)
    svc_b = PrivacyService(key_b)
    encrypted = svc_a.encrypt("body")
    with pytest.raises(StoragePersistenceError):
        svc_b.decrypt(encrypted) # wrong key must raise, not return garbage

def test_is_available_returns_true_when_key_set(privacy):
    assert privacy.is_available() is True

def test_is_available_returns_false_when_no_key():
    svc = PrivacyService(key=None)
    assert svc.is_available() is False

```

3. Requirements and User Story Alignment

Test File	Test Name	Req ID	User Story	What It Verifies
test_note_creation	test_empty_title_raises_validation_error	FR-01	US-01 AC: empty title rejected	ValidationLayer rejects empty input before any persistence
test_note_creation	test_whitespace_only_title_raises_validation_error	FR-01 refined (Week 3.1 A-01)	US-01 AC: whitespace-only title rejected	Edge case from ambiguity review: ' ' treated as empty
test_note_creation	test_create_note_assigns_utc_timestamps	FR-06 refined	US-01 AC: timestamps set automatically	Timestamps are UTC as required by Week 3.1 refinement
test_note_creation	test_create_note_persists_json_file	FR-05	US-05 AC: note survives session	JsonFileRepository writes a recoverable JSON file to disk
test_note_creation	test_create_note_tags_stored_as_entered	FR-06	US-08 AC: tags case-sensitive	Tags are case-sensitive per refined FR-06
test_secure_note	test_encrypt_decrypt_round_trip	FR-04, SPR-01	US-04 AC: decrypted body matches original	PrivacyService encrypt/decrypt are exact inverses
test_secure_note	test_encrypted_output_is_not_plaintext	SPR-01	US-04 AC: body never stored as plaintext	Core SPR-01 guarantee: no plaintext on disk for private notes
test_secure_note	test_wrong_key_raises_storage_error	FR-07 refined (Week 3.1 gap)	US-04 AC: decrypt failure surfaces error	Wrong key raises StoragePersistenceError, not silent garbage

4. Test Levels

Test	Level	Why This Level
test_empty_title / test_whitespace_only_title	Unit	Tests one rule in ValidationLayer with no dependencies. Fastest possible feedback.
test_create_note_assigns_uuid / test_create_note_assigns_utc_timestamps	Unit	Tests Note dataclass construction in isolation. No repo or service needed.
test_create_note_persists_json_file	Integration	Tests NoteService + ValidationLayer + JsonFileRepository working together. Uses tmp_path, not a mock.
test_create_note_tags_stored_as_entered	Integration	Tests that the full create path preserves tags exactly as entered across serialization.
test_encrypt_decrypt_round_trip / test_encrypted_output_is_not_plaintext	Unit	Tests PrivacyService in isolation with a test key. Core SPR-01 guarantee.
test_wrong_key_raises_storage_error	Unit	Tests PrivacyService error path. Wrong key must raise, not return garbage content.
test_is_available_returns_false_when_no_key	Unit	Tests PrivacyService availability check. Guards against using encryption without a key.

5. When These Tests Run

Test Group	When It Runs	Reason
All unit tests (ValidationLayer, Note, PrivacyService)	On every file save / every commit	Sub-second runtime. Catch logic errors immediately before they compound.
Integration tests (NoteService + JsonFileRepository)	On every commit and every sprint boundary	Slightly slower due to tmp_path I/O. Worth running on every commit given the low count.
SPR-01 encryption tests (test_secure_note.py)	On every commit touching privacy or repository layer	Privacy is a first-class requirement. Any change to PrivacyService or JsonFileRepository must re-verify the no-plaintext guarantee.
Feature / CLI tests (not yet written)	At milestone delivery and before demo	End-to-end tests are slower and more brittle. They validate the full user story at delivery time, not during daily development.

6. How AI Helped Draft and Critique the Tests

What AI helped generate:

An initial list of test cases for note creation. AI suggested the UUID, timestamp, and persistence tests correctly. These were accepted with minor additions for UTC timezone checking.

The round-trip encrypt/decrypt test and the plaintext-check test for PrivacyService. Both were structurally correct and accepted.

A suggestion to add a test for creating a note with a very long title. Evaluated and rejected as low-value for this sprint since no maximum title length requirement exists.

A suggestion to test that list_all() returns notes in insertion order. Rejected because FR-08 explicitly defines modified_at ordering. Insertion order is undefined and testing for it would be testing the wrong thing.

What was changed or rejected:

AI generated a test named `test_create_note_succeeds` that only asserted the return value was not `None`. Rejected as a noise test: it tests that Python returned an object, not that the object is correct.

AI did not include the whitespace-only title test. Added manually based on the Week 3.1 ambiguity review (A-01). This is the most important edge case for FR-01 and AI missed it because it had no context for the refined requirement.

AI suggested using `unittest.mock` to mock `JsonFileRepository` in the integration tests. Rejected: using a real `tmp_path` directory gives more honest coverage for FR-05 and avoids mocking what should actually be tested.

AI generated a test for `test_wrong_key_raises_error` but used a bare `Exception` catch rather than `StoragePersistenceError`. Changed to assert the specific exception type, which validates that the error handling in `PrivacyService` matches the class diagram.